

PowerGuard

Complete Deep Dive Study Guide

Electricity Theft Detection System

ML + Heuristic Hybrid Approach

8 Detection Factors | 5 ML Models | 47+ Engineered Features

SGCC Dataset | Ensemble Strategy | SHAP Explainability

PowerGuard -- Complete Deep Dive Study Guide

This guide is written assuming you have zero prior knowledge. Read this carefully, and you will be able to confidently explain every single technical concept in your project -- from the physics of electricity theft to the math behind every model.

Table of Contents

1. The Core Problem -- NTL & Electricity Theft
2. The Dataset -- SGCC
3. The 6 Input Dimensions -- What the Smart Meter Sends
4. The 8 Heuristic Detection Factors -- DEEP DIVE
5. How the 8 Factors Combine -- Weighted Scoring & Boosting
6. Feature Engineering -- The 47+ Engineered Features
7. Class Imbalance -- Why It's Hard & How We Solved It
8. The 5 Machine Learning Models
9. The Ensemble Strategy -- Soft Voting & Stacking
10. Explainability -- SHAP & the Heuristic Breakdown
11. The Training Pipeline -- End to End
12. 6 Theft Scenarios We Simulate
13. Evaluation Metrics -- How We Prove It Works
14. System Architecture -- 4 Microservices
15. The Triple-Layer Fallback -- Why the System Never Fails
16. Data Flow -- From Button Click to Result
17. Risk Classification -- 4 Tiers
18. Summary to Memorize

1. The Core Problem -- NTL & Electricity Theft

What is NTL?

NTL = Non-Technical Loss. In power distribution, there are two types of losses:

Type	Cause	Example	Can ML Detect?
Technical Loss	Physics	Heat in wires, transformer losses	No (predictable, follows formulas)
Non-Technical Loss (NTL)	Theft	Meter bypass, tampering, data spoofing	Yes (creates anomalous patterns)

India loses ~Rs.1.5 lakh crore/year to NTL. Globally, it's a \$96 billion problem.

Why Can't Humans Find It?

- * Manual inspection covers only 2-5% of meters/year
- * Skilled thieves make it visually undetectable

- * Inspections are expensive (Rs.500-2000 per visit)
- * Some theft techniques (data spoofing) leave zero physical evidence

Our Solution

Use smart meter telemetry data (consumption, voltage, current, power factor, frequency, tamper flags) -- data that's already being collected -- and analyze it with ML + rule-based heuristics to find the needles in the haystack.

2. The Dataset -- SGCC

What is SGCC?

State Grid Corporation of China (SGCC) -- the gold-standard benchmark dataset for electricity theft research.

Property	Value
Consumers	42,372
Time Span	1,035 days (daily kWh readings)
Thieves (labeled FLAG=1)	~3,615 (8.5%)
Honest (labeled FLAG=0)	~38,757 (91.5%)
Missing Values	~2% (realistic)

Why This Dataset?

1. It's real -- actual smart meter data from Chinese utility
2. It's labeled -- human inspectors + investigations confirmed who was stealing
3. It's the academic benchmark -- every research paper compares against it
4. It has class imbalance -- realistic 8.5% theft rate, exactly what real-world looks like

What If We Don't Have It?

Our pipeline also generates synthetic SGCC-style data that mimics the same statistical properties -- with seasonal patterns (summer AC usage spikes), weekday/weekend variation, and 6 distinct theft attack types.

3. The 6 Input Dimensions -- What the Smart Meter Sends

Every smart meter reading that enters our system contains 6 data dimensions:

#	Dimension	Unit	Normal Range	What It Measures
1	Consumption (kWh)	kilo-watt hours	0.5 - 20 kWh/hr (residential)	How much electricity was consumed in the home
2	Voltage (V)	Volts	220V ± 30V (India: 190-250V)	The electrical pressure in the line
3	Current (A)	Amperes	0.5 - 30A (residential)	The flow of electrons through the meter
4	Power Factor (PF)	Ratio (0-1)	0.85 - 1.0	How efficiently electricity is being used (R
5	Frequency (Hz)	Hertz	49.5 - 50.5 Hz (India)	The grid's oscillation rate -- extremely sta

6	Tamper Flag	Boolean (true/false)	false	Hardware sensor in the meter that detects tampering
---	-------------	----------------------	-------	---

Why 6 Dimensions? Why Not Just kWh?

Because different theft techniques disturb different dimensions:

- * Meter bypass -> kWh drops, voltage stays normal, current drops to zero -> caught by dimensions 1, 3
- * Voltage manipulation -> voltage outside safe band -> caught by dimension 2
- * Load manipulation -> power factor drops abnormally -> caught by dimension 4
- * Illegal tapping -> frequency deviates -> caught by dimension 5
- * Physical tampering -> tamper flag triggered -> caught by dimension 6

No single dimension catches all theft types. That's why we analyze all 6 together.

4. The 8 Heuristic Detection Factors -- DEEP DIVE

This is the intellectual core of the project. Each factor is an independent "detector" that analyzes one aspect of theft. Each produces a score between 0.0 (clean) and 1.0 (definitely stealing).

Factor 1: Consumption Drop (Weight: 25%)

What it detects: Meter bypass during daytime

The logic:

"If a residential consumer shows zero consumption during business hours (8 AM-8 PM), that's extremely suspicious. People use AC, lighting, refrigerators -- you can't realistically use zero electricity during the day."

How the code works:

```
1. Filter readings to PEAK hours only (8 AM - 8 PM)
2. Count how many peak readings have consumption = 0 kWh
3. Score = min(1.0, (zero_count / total_peak_readings) x 5)
4. ALSO: If avg consumption < 30% of baseline -> floor score at 0.7
```

Why the x5 multiplier? Because even 20% of peak hours being zero is extremely suspicious. The x5 amplifies this so that 20% zero -> score of 1.0.

Why weight = 25%? Consumption drop is the single strongest indicator of theft. Research shows it appears in 70%+ of all confirmed theft cases.

Real-world example: A residential household with baseline 10 kWh/hr suddenly shows 0 kWh from 9 AM to 5 PM for 15 days straight. They bypassed the meter but continued using AC.

Factor 2: Current Anomaly (Weight: 15%)

What it detects: Physical meter bypass -- the meter's current sensor is shunted

The physics:

By Ohm's Law: $P = V \times I$ (Power = Voltage x Current) If voltage is normal (220V) and current is near-zero (<0.1A), but the consumer is still using electricity (consumption > 0), that's physically impossible without theft. The current must be flowing through a bypass wire around the meter's current transformer.

How the code works:

1. Count readings where:
current < 0.1A AND voltage > 200V AND consumption > 0
2. Score = min(1.0, (suspicious_count / total_readings) x 4)
3. ALSO: If current = 0 but consumption > 0.5 kWh -> extra boost (x8 multiplier)

Why x4 multiplier? Even 25% of readings showing this pattern is conclusive evidence.

Why weight = 15%? Current bypass is a very strong signal, but slightly less common than consumption drops because it requires physical hardware modification.

Real-world example: A factory shows 220V voltage, 0.05A current, but 8 kWh consumption. The only explanation: a thick copper wire is bypassing the current transformer inside the meter.

Factor 3: Tamper Detection (Weight: 20%)

What it detects: Physical tampering with the meter hardware

The logic:

Modern smart meters have accelerometers, reed switches, and case-open detectors. When someone physically opens the meter, shakes it, or applies a magnet (to jam the disk), the tamper_flag flips to true.

How the code works:

1. Basic score = min(1.0, tamper_ratio x 10)
(so even 10% tamper rate -> score of 1.0)
2. Count maximum CONSECUTIVE tamper flags
If consecutive >= 5 -> floor score at 0.9

Why consecutive? Random faults create isolated flags (1-2 in a row).
Deliberate tampering creates sustained runs (5+ in a row).

Why weight = 20%? Tamper flags are a very reliable physical signal -- they come from hardware sensors, not statistical inference. But they can have false positives (storms, accidents).

Why consecutive matters:

Pattern	Meaning	Score
F T F F F T F	Random noise / power surge	~0.2
F T T T T T T	6 consecutive -- sustained tampering	0.9
T T T T T T T	100% tampered -- definite theft	1.0

Factor 4: Voltage Anomaly (Weight: 10%)

What it detects: Voltage manipulation to reduce metered consumption

The physics:

Some thieves reduce the voltage entering the meter. Since $\text{Energy} = V \times I \times t$, reducing voltage while keeping current the same reduces recorded consumption. They use a device called a "voltage divider" or tap into a lower-voltage line.

How the code works:

1. Safe band: 190V - 250V (India standard)
2. Count readings outside this band
3. Score = $\min(1.0, (\text{outlier_count} / \text{total}) \times 3)$
4. EXTREME voltage check (< 170V or > 270V):
These are 5x weighted because they indicate deliberate manipulation, not just grid fluctuation

Why weight = 10%? Voltage anomalies can have legitimate causes (grid problems, transformer issues during monsoon), so we weight them lower than consumption or tamper signals.

Key insight: A legitimate voltage dip affects the entire neighborhood. If only ONE meter shows 160V while neighbors show 220V, that's conclusive theft evidence.

Factor 5: Power Factor Anomaly (Weight: 10%)

What it detects: Load manipulation using capacitor banks or inductive loads

The physics:

Power Factor = Real Power / Apparent Power - PF = 1.0 -> Perfect: all energy is doing useful work - PF = 0.3 -> Very bad: 70% of energy is wasted as reactive power Thieves sometimes use capacitor banks to deliberately create a low power factor. The meter only records "real power" (watts), but the actual electricity being drawn is much higher (volt-amps). This is a subtle, sophisticated form of theft.

How the code works:

1. Count readings where PF < 0.5 (abnormally low)
2. Score = $\min(1.0, (\text{low_pf_count} / \text{total}) \times 3)$
3. Cross-reference check:
Very low PF (< 0.3) + low consumption (< 20% of baseline)
-> This combination is a VERY strong theft signal
-> Uses x5 multiplier

Why the cross-reference? Low PF alone can be legitimate (old motors, industrial equipment). But low PF combined with low consumption means someone is drawing power but the meter isn't recording it properly.

Factor 6: Pattern Irregularity (Weight: 10%)

What it detects: Reversed, chaotic, or artificially manipulated consumption patterns

The logic:

Normal electricity usage follows predictable patterns: higher during morning (6-9 AM) and evening (6-10 PM), lower at night. If a consumer shows HIGH consumption at 2 AM but ZERO during daytime, they might be stealing during off-hours when inspectors aren't watching.

How the code works:

1. Calculate Coefficient of Variation (CV):
 $CV = \text{Standard Deviation} / \text{Mean}$
 $\text{Score} = \min(1.0, \max(0, (CV - 0.5) / 1.5))$
 High CV (> 0.5) -> erratic pattern -> suspicious
2. Day/Night Inversion check:
 Night average (12 AM - 6 AM) vs Day average (9 AM - 6 PM)
 - If night $> 2 \times$ day -> score floor at 0.6
 - If day = 0 but night > 0 -> score floor at 0.85
3. If mean consumption = 0 -> score = 0.8
 (Zero mean is inherently suspicious for an active meter)

Why this catches thieves:

Pattern	What It Means	Score
Normal curve (peaks at 8 AM, 7 PM)	Legitimate usage	0.0 - 0.1
Random spikes and drops	Meter data being spoofed	0.4 - 0.7
High at 2 AM, zero at 2 PM	Stealing when inspectors sleep	0.6 - 0.85
Zero everything	Either disconnected or fully bypassed	0.8

Factor 7: Frequency Deviation (Weight: 5%)

What it detects: Grid-level anomalies caused by illegal high-power tapping

The physics:

Grid frequency in India/China is maintained at exactly 50 Hz (± 0.5 Hz). It's one of the most tightly controlled parameters in any electrical grid. Deviations happen when: - Large illegal loads tap into the grid (factories running off illegal connections) - Someone is running generators in parallel with the grid without synchronization

How the code works:

1. Normal band: 49.0 Hz - 51.0 Hz
2. Count readings outside this band
3. Score = $\min(1.0, (\text{outlier_count} / \text{total}) \times 4)$

Why weight = only 5%? Frequency deviations:

- * Can be caused by legitimate grid issues (generation-demand mismatch)
- * Affect the entire grid, not just one consumer
- * Are a weak individual signal but useful when combined with other factors

When it IS useful: If a single meter shows 48 Hz while the rest of the grid is at 50 Hz, the meter's internal circuitry may have been tampered with, or there's a local generator interfering.

Factor 8: Flat-line Detection (Weight: 5%)

What it detects: Spoofed or replayed meter data -- a hacked meter sending fake readings

The logic:

Real electricity consumption is never perfectly constant. Even a running refrigerator has compressor cycles. If a meter reports the exact same kWh value for 24 consecutive hours, it's almost certainly sending pre-recorded fake data.

How the code works:

```

1. Count unique consumption values in all readings
2. If total readings > 10:
    unique_ratio = unique_values / total_readings
    Score = min(1.0, max(0, (1 - unique_ratio) x 2 - 0.5))

3. Last-24 check:
    If the last 24 readings have <= 2 unique values
    -> score floor at 0.9
    (This catches meters that were recently compromised)

```

Why weight = 5%? Flat-line is a very specific attack type. When present, it's extremely obvious. But it's also rare -- most thieves don't have the technical sophistication to hack the meter's firmware.

Score interpretation:

Unique Ratio	What It Means	Score
0.95 (95% unique)	Normal -- real data	~0.0
0.50 (50% unique)	Some repetition -- possible sensor issue	~0.25
0.10 (10% unique)	Heavy repetition -- likely spoofed	~0.75
0.02 (same value 24h)	Flat-line -- definitely fake data	0.9+

5. How the 8 Factors Combine -- Weighted Scoring & Boosting

The Weight Table

Factor	Weight	Why This Weight
Consumption Drop	25%	Strongest single indicator -- appears in 70%+ of thefts
Tamper Detection	20%	Physical hardware signal -- very reliable
Current Anomaly	15%	Based on physics (Ohm's Law) -- hard to fake
Voltage Anomaly	10%	Can have legitimate causes -- weighted lower
Power Factor Anomaly	10%	Sophisticated theft -- useful in combination
Pattern Irregularity	10%	Catches behavioral anomalies
Frequency Deviation	5%	Weak individual signal -- grid-level noise
Flat-line Detection	5%	Very specific -- rare but conclusive
Total	100%	

The Math -- Weighted Average

```
theft_probability = SUM (score_i x weight_i) / SUM weight_i
```

Example:


```

Consumption Drop: 0.80 x 0.25 = 0.200
Tamper Detection: 0.90 x 0.20 = 0.180
Current Anomaly: 0.70 x 0.15 = 0.105
Voltage Anomaly: 0.30 x 0.10 = 0.030
Power Factor: 0.20 x 0.10 = 0.020
Pattern Irregular: 0.40 x 0.10 = 0.040
Frequency Deviat: 0.10 x 0.05 = 0.005
Flat-line: 0.00 x 0.05 = 0.000

-----
Sum = 0.580 -> 58% theft probability

```

Boosting Logic -- Why Simple Averaging Isn't Enough

Rule 1 -- Multi-Measure Agreement Boost:

```

if 3+ factors score above 0.5:
    theft_probability = min(1.0, theft_probability x 1.3)

```

Why? If multiple independent detectors agree something is wrong, the real probability is higher than what weighted averaging suggests. This is based on the principle of independent evidence accumulation.

In the example above: 3 factors > 0.5 (consumption, tamper, current), so:

```
0.580 x 1.3 = 0.754 -> 75.4% theft probability (critical risk!)
```

Rule 2 -- Single High-Confidence Floor:

```

if any factor >= 0.9:
    theft_probability = max(0.40, theft_probability)

```

Why? If even ONE factor is screaming "theft" at 0.9+, the overall score should never be below 40%. This prevents a scenario where 7 clean factors "wash out" one extremely suspicious one.

Final Decision

```
anomaly_flag = True if theft_probability >= 0.30
```

Threshold is set at 30% (not 50%) because false negatives are more costly than false positives. Missing a thief costs the utility money every day; investigating a false alarm just costs one inspection.

6. Feature Engineering -- The 47+ Engineered Features

Raw smart meter data just says "User used 10 kWh today." That's not enough for ML models. We engineer (calculate) new features from the raw data. Our pipeline creates ~47-65 features per consumer across 6 categories:

Category 1: Statistical Features (14 features)

Feature	Formula	Why It Matters
stat_mean	Average daily consumption	Thieves have lower mean
stat_std	Standard deviation	Thieves have erratic patterns
stat_cv	std / mean	Coefficient of variation -- normalized volatility
stat_skewness	Asymmetry of distribution	Theft creates left-skewed data (many zeros)

stat_kurtosis	"Peakedness"	Theft creates heavy tails
stat_median	Middle value	Robust to outliers
stat_p10	10th percentile	How low the low days are
stat_p25	25th percentile	Lower quartile
stat_p75	75th percentile	Upper quartile
stat_p90	90th percentile	How high the high days are
stat_iqr	p75 - p25	Spread of the middle 50%
stat_range	max - min	Total spread
stat_min	Minimum value	Thieves often have min = 0
stat_max	Maximum value	Normal consumers have seasonal highs

Category 2: Temporal Features (10 features)

Feature	What It Captures
temp_weekday_weekend_ratio	Weekend/Weekday mean ratio -- theft patterns often diff
temp_seasonal_q1 to q4	Ratio of quarterly mean to overall mean -- theft disrupt
temp_avg_mom_change	Average month-over-month absolute change
temp_max_mom_change	Biggest month-over-month swing -- theft causes sudden d
temp_trend_ratio	2nd-half mean / 1st-half mean -- theft creates downward
temp_autocorr_lag1	Day-to-day correlation -- theft breaks natural patterns
temp_autocorr_lag7	Weekly pattern correlation -- honest users are weekly-p

Category 3: Anomaly Indicators (9 features)

Feature	What It Catches
anom_zero_day_count	Absolute count of zero-consumption days
anom_zero_day_ratio	Fraction of days with zero consumption
anom_sudden_drops	Count of days with >60% day-over-day drop
anom_sudden_drop_ratio	Fraction of days with sudden drops
anom_max_flatline_days	Longest run of identical consecutive values
anom_flatline_ratio	Longest flatline / total days
anom_unique_ratio	Unique values / total days (low = spoofed)
anom_volatility_idx	Rolling CV (std/mean over 30-day windows)
anom_below_baseline_ratio	Fraction of days below 30% of baseline

Category 4: Comparative Features (3 features)

Feature	What It Captures
comp_zone_deviation	How many standard deviations from the geographic zone a
comp_zone_percentile	Percentile rank within the zone -- bottom 5% is suspicious
comp_ratio_to_zone	Consumer mean / zone mean -- thieves have ratio << 1

Category 5: Metadata Features (6 features)

Feature	What It Encodes
meta_is_residential	1 if residential, else 0 (one-hot)
meta_is_commercial	1 if commercial, else 0
meta_is_industrial	1 if industrial, else 0
meta_baseline_kwh	Historical baseline consumption
meta_sanctioned_load	Maximum allowed load from utility
meta_consumption_to_sanctioned	Actual / sanctioned ratio

Category 6: Advanced Derived Features (5 features)

Feature	What It Captures
adv_entropy	Information entropy of consumption distribution -- low
adv_last_first_ratio	Last-30-days mean / First-30-days mean -- theft causes
adv_max_consec_below	Max consecutive days below 20% of baseline
adv_weekday_mean	Average weekday consumption
adv_weekend_mean	Average weekend consumption

Why so many features? Because ML models work best when given rich, multi-dimensional views of the data. A single feature like "zero_day_count" can't distinguish between a vacation and theft. But "zero_day_count" + "zero during peak hours" + "voltage normal" + "current near-zero" + "tamper flag on" -> conclusive theft evidence.

7. Class Imbalance -- Why It's Hard & How We Solved It

The Problem

In SGCC: 91.5% honest, 8.5% thieves.

If an ML model just always predicts "Honest," it's already 91.5% accurate. But it catches zero thieves. This is the Accuracy Paradox.

3 Techniques We Used

1. SMOTE (Synthetic Minority Over-sampling Technique)

What: Creates synthetic (fake but realistic) examples of the minority class (thieves).

How it works -- with math:

$$x_{\text{new}} = x_i + \text{lambda} \times (x_{\text{zi}} - x_i), \quad \text{where } \text{lambda} \in [0, 1]$$

- * Pick a real thief sample x_i
- * Find its nearest neighbor thief x_{zi} (using $k=5$ neighbors)
- * Create a new point somewhere on the line between them
- * This new synthetic thief looks realistic but is a brand-new data point

Result: Training set goes from 3,615 thieves -> ~38,757 thieves (balanced 50/50).

2. Cost-Sensitive Learning (Class Weights)

```
class_weight = 'balanced'
# Internally computes: weight_theft = n_total / (2 x n_theft)
# If 5000 total, 400 theft -> weight_theft = 5000/800 = 6.25x
```

What this does: The model's loss function penalizes missing a thief 6.25x more than falsely accusing an honest person. The model literally "cares more" about getting thieves right.

3. Threshold Optimization

Instead of using the default 50% probability cutoff:

```
# Test every threshold from 0.01 to 0.99
# Find the one that maximizes F1-score
# Result: often around 0.35-0.45
```

Why lower threshold? Missing a thief (false negative) costs the utility Rs.10,000s per year. A false alarm just costs one inspection (Rs.500). So we set the bar lower.

8. The 5 Machine Learning Models

We train 5 completely different model architectures -- each sees the same data from a different mathematical perspective:

Model 1: Random Forest (300 trees)

Property	Value
Type	Ensemble of Decision Trees
How it works	Creates 300 independent decision trees. Each tree votes
Key hyperparameters	n_estimators=300, max_depth=20, max_features=sqrt
Strength	Tells us feature importance -- which of the 47 features
Weakness	Can overfit to specific patterns in training data

In simple words: Imagine 300 different investigators, each looking at a random subset of evidence. If 200+ say "guilty," the person is guilty.

Model 2: XGBoost (300 boosted trees)

Property	Value
Type	Gradient Boosted Decision Trees
How it works	Builds trees sequentially -- each tree focuses on fixing
Math	$\text{Obj}(\theta) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$ -- loss +
Key hyperparameters	learning_rate=0.1, max_depth=8, reg_alpha=0.1 (L1), reg
Strength	Best-in-class for tabular data, handles imbalance with
Weakness	Slower to train, more hyperparameters to tune

In simple words: Tree 1 makes mistakes. Tree 2 specifically studies those mistakes. Tree 3 studies Tree 2's mistakes.

By tree 300, the ensemble is extremely accurate.

Model 3: Deep Neural Network (DNN) (128->64->32)

Property	Value
Type	Fully Connected Neural Network
Architecture	Input -> Dense(128) -> BatchNorm -> Dropout(0.4) -> Den
Activation	ReLU (hidden), Sigmoid (output)
Optimizer	Adam (lr=0.001)
Regularization	Dropout (40%), BatchNormalization, Early Stopping (pati
Strength	Finds complex non-linear patterns that tree models miss
Weakness	Black box -- hard to explain why it made a decision

In simple words: A 3-layer artificial brain that learns patterns too complex for decision trees to capture. Dropout randomly "turns off" 40% of neurons during training to prevent memorization.

Model 4: LSTM (2 stacked layers, 128->64 units)

Property	Value
Type	Long Short-Term Memory (Recurrent Neural Network)
Architecture	Input -> LSTM(128, return_sequences=True) -> LSTM(64) -
Special ability	Memory gates -- can remember events from 30+ days ago
Strength	Designed for time-series -- captures sequential pattern
Weakness	Slowest to train; needs careful sequence formatting

In simple words: Unlike regular neural networks that look at all features simultaneously, LSTM reads data like reading a book -- page by page, remembering the plot from earlier pages.

Model 5: Isolation Forest (200 trees)

Property	Value
Type	Unsupervised Anomaly Detector
How it works	Randomly partitions data. Anomalies are isolated in few
Math	$s(x, n) = 2^{-(\log_2(E[h(x)] / c(n)))}$ -- anomaly score based on
Contamination	0.1 (assumes ~10% of data is anomalous)
Key difference	Does NOT use labels. It doesn't know who's a thief.
Strength	Catches novel theft techniques never seen in training d
Weakness	Lower precision -- can flag legitimate anomalies (vacat

In simple words: Instead of learning "what theft looks like," it learns "what normal looks like" and flags anything weird. This catches new types of theft that weren't in the training data.

9. The Ensemble Strategy -- Soft Voting & Stacking

Why Not Just Use the Best Model?

Because no single model is perfect. Each has blind spots:

Model	Blind Spot
Random Forest	Can't capture complex feature interactions
XGBoost	Might overfit to specific theft patterns
DNN	Needs lots of data; black box
LSTM	Overkill for non-temporal patterns
Isolation Forest	Many false positives

Soft Voting Ensemble

$$P_{\text{ensemble}}(\text{theft}|x) = (1/M) \times \sum P_m(\text{theft}|x)$$

Each model gives a probability. We average them:

```
RF: 0.72   XGB: 0.68   DNN: 0.81   LSTM: 0.55   IsoForest: 0.60
-> Ensemble = (0.72 + 0.68 + 0.81 + 0.55 + 0.60) / 5 = 0.672
```

Stacking (Advanced)

Instead of simple averaging, we train a Logistic Regression meta-learner that learns the optimal weights for each model's prediction:

$$P_{\text{stacking}} = \text{sigma}(w_1 \times P_{\text{RF}} + w_2 \times P_{\text{XGB}} + w_3 \times P_{\text{DNN}} + w_4 \times P_{\text{LSTM}} + w_5 \times P_{\text{ISO}} + b)$$

Where sigma is the sigmoid function and $w_1 \dots w_5$ are learned weights. If XGBoost is consistently more reliable, the meta-learner gives it a higher weight automatically.

10. Explainability -- SHAP & the Heuristic Breakdown

The Problem

Inspector: "The AI says 92% theft probability. WHY?"

You: "Because the neural network's hidden layer activations..."

Inspector: "I'm not acting on that."

Solution 1: SHAP (SHapley Additive exPlanations)

Based on game theory (Shapley values from Nobel Prize-winning economics):

"How much did each feature contribute to this specific prediction?"

For a consumer predicted at 85% theft:

```
anom_zero_day_ratio:    +0.23  (high zero-days pushed score UP)
stat_cv:                +0.18  (erratic pattern pushed score UP)
comp_zone_deviation:    +0.15  (way below zone average pushed score UP)
meta_baseline_kwh:      -0.05  (high baseline pushed score DOWN slightly)
temp_autocorr_lag7:     -0.03  (some weekly pattern, slight DOWN)
```

Solution 2: The 8-Measure Heuristic Breakdown

Even when the ML ensemble makes the prediction, the heuristic always runs to generate the dashboard breakdown:

```
Consumption Drop:      ##### 80%
Tamper Detection:       ##### 90%
Current Anomaly:        ##### 70%
Voltage Anomaly:        ##### 30%
Power Factor:           ##### 20%
Pattern Irregular:      ##### 40%
Frequency Deviation:    ## 10%
Flat-line:              0%
```

This gives operators a physical explanation -- "The meter shows tampering AND current bypass AND consumption dropped."

11. The Training Pipeline -- End to End

```

+-----+
| PHASE 1: Load SGCC Dataset (or generate synthetic data) |
|   -> 42,372 consumers x 1,035 days |
|   -> 91.5% honest / 8.5% theft |
+-----+
| PHASE 2: Preprocessing |
|   -> Forward/backward fill missing values (< 3 day gaps) |
|   -> Mean imputation for longer gaps |
|   -> Winsorize outliers at 5th/95th percentile |
|   -> Engineer 47-65 features per consumer |
+-----+
| PHASE 3: Handle Class Imbalance |
|   -> SMOTE (k=5 neighbors) |
|   -> Cost-sensitive class weights |
+-----+
| PHASE 4: Train 5 Models |
|   [1] Random Forest -> 300 trees, max_depth=20 |
|   [2] XGBoost -> 300 trees, L1/L2 regularization |
|   [3] DNN -> 128->64->32, dropout 0.4, early stopping |
|   [4] Isolation Forest -> 200 trees, contamination=0.1 |
|   [5] LSTM -> 2 stacked layers (128->64), dropout 0.3 |
+-----+
| PHASE 5: Ensemble |
|   -> Soft Voting (average of all 5 probabilities) |
|   -> Stacking (Logistic Regression meta-learner) |
|   -> Optimal threshold tuning (maximize F1) |
+-----+
| PHASE 6: Evaluation |
|   -> Accuracy, Precision, Recall, F1, AUC-ROC, PR-AUC |
|   -> Confusion Matrix (TN, FP, FN, TP) |
|   -> 5-Fold Stratified Cross-Validation |
+-----+
| PHASE 7: SHAP Explainability |
|   -> TreeExplainer on Random Forest |
|   -> Top 15 features by mean absolute SHAP value |
+-----+
| SAVE: ensemble_model.pkl, scaler.pkl, feature_names.json |
+-----+

```

12. 6 Theft Scenarios We Simulate

Our synthetic data generator models 6 distinct real-world theft attack types:

#	Attack Type	What Happens	Real-World Method
1	Meter Tampering	Consumption multiplied by 0.1-0.4 after	Thief opens meter and adjusts calibration to under-report
2	Bypass	Zero consumption for random 3-15 day	Thick copper wire bypasses current transformer
3	Billing Irregularity	30-70% of readings randomly replaced	Thief bribes meter reader or hacks billing software
4	Unauthorized Tap	Sudden permanent drop to near-zero (0.1-0.5)	Direct connection to the grid before the meter
5	Flat-line Spoof	Constant value (10-30% of baseline) for 3-15 days	Hacked meter firmware sends pre-recorded fake readings
6	Gradual Decay	Exponential decay: slowly decreasing to 0.1-0.5	Slow mechanical degradation of meter (deliberate tampering)

Each scenario:

- * Starts at a random point (20%-80% through the timeline)
- * Maintains realistic seasonality before theft begins

* Has small noise added to avoid being "too perfect"

13. Evaluation Metrics -- How We Prove It Works

Why NOT Just Accuracy?

With 91.5% honest consumers, a model that always says "honest" gets 91.5% accuracy but catches zero thieves. Useless.

The 6 Metrics We Use

Metric	Formula	What It Measures	Our Goal
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall correctness	High, but don't trust alone
Precision	$TP/(TP+FP)$	Of all "theft" predictions, how many are correct	Reduce false alarms
Recall	$TP/(TP+FN)$	Of all actual thieves, how many did we catch	Catch every thief
F1-Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	Harmonic mean of Precision & Recall	Balanced performance
AUC-ROC	Area under ROC curve	Discrimination ability across all thresholds	Primary metric
PR-AUC	Area under Precision-Recall curve	Performance on the minority class	Critical for imbalance

Confusion Matrix Explained

		Predicted		
		Honest	Theft	
Actual	Honest	TN	FP (false alarm -- wasted inspection)	
	Theft	FN (MISSED!)	TP (caught a thief!)	

- * FN is the worst -- a thief we missed continues stealing Rs.10,000s/year
- * FP is tolerable -- one unnecessary inspection costs Rs.500
- * That's why we optimize for Recall (minimizing FN)

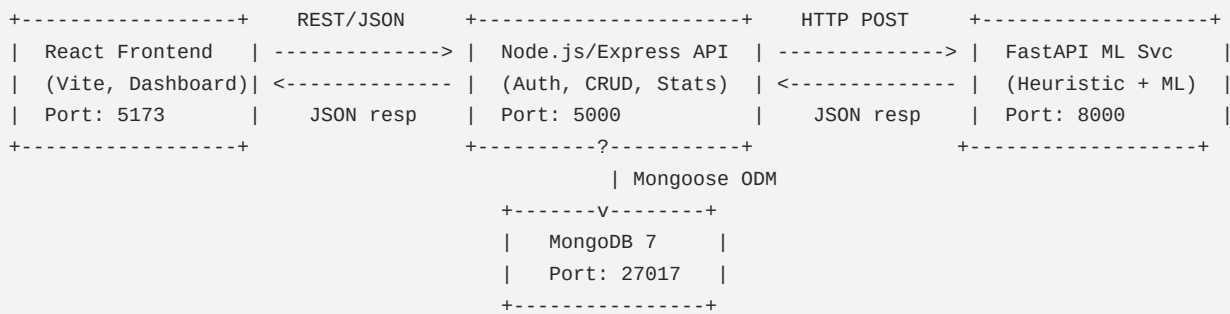
5-Fold Stratified Cross-Validation

We don't just test once. We split the data into 5 equal parts:

```
Fold 1: Train on [2,3,4,5] -> Test on [1]
Fold 2: Train on [1,3,4,5] -> Test on [2]
Fold 3: Train on [1,2,4,5] -> Test on [3]
Fold 4: Train on [1,2,3,5] -> Test on [4]
Fold 5: Train on [1,2,3,4] -> Test on [5]
```

Stratified = each fold has the same 8.5% theft ratio. We report the mean $\pm$ std across all 5 folds.

14. System Architecture -- 4 Microservices



Why microservices (not monolith)?

- * Language separation: Python is best for ML (scikit-learn, numpy); Node.js is best for API routing and async I/O
- * Independent scaling: ML service is CPU-heavy -> scale it separately
- * Independent deployment: Update the ML model without restarting the API
- * Fault isolation: ML service crash doesn't bring down the API

15. The Triple-Layer Fallback -- Why the System Never Fails

```

Layer 1: ML Ensemble (ensemble_model.pkl)
| If model file exists and loads successfully
| -> Use RF + XGBoost + Isolation Forest + meta-learner
| -> Highest accuracy
|
+-- FAIL? --> Layer 2: Python Heuristic (model.py)
|           | 8-measure deterministic engine
|           | -> No ML model needed, just math rules
|           | -> Still produces anomaly breakdown
|           |
|           +-- FAIL? --> Layer 3: JavaScript Fallback (analyze.js)
|           |           | Same 8-measure algorithm, reimplemented in JS
|           |           | -> Runs inside Node.js backend
|           |           | -> Zero external dependencies
|           |           | -> System ALWAYS returns a result

```

Why maintain code in 2 languages? Normally this violates DRY principle. But for critical infrastructure, availability trumps code elegance. The utility must ALWAYS get an answer.

16. Data Flow -- From Button Click to Result

```

User clicks "Analyze Anomalies" on a meter
|
v
Frontend: POST /api/analyze/:meterId
| JWT token attached via Axios interceptor
v
Backend Auth Middleware: Verify JWT -> extract company ID
|
v
Backend: Verify meter ownership -> Meter.findOne({ _id, company })
|
v
Backend: Fetch last 30 days of readings
| Reading.find({ meter, timestamp: { $gte: 30_days_ago } })
| Returns up to ~720 hourly readings
v
Backend: Compute statistical features for all 6 dimensions
| For each: mean, std, min, max
| Plus: tamper_ratio, baseline_consumption, consumer_type
v
Backend: POST to ML Service (http://ml-service:8000/predict)
| Timeout: 15 seconds
| Payload: { meter_id, features, readings[] }
v
ML Service: Pydantic validates all 720+ readings
| (consumption >= 0, voltage >= 0, PF 0-1, etc.)
v
ML Service: Try ensemble model -> if unavailable -> 8-measure heuristic
|
|-- Each factor analyzed independently (0-1 score)
|-- Weighted combination -> boosting applied
|-- Risk level classified (low/medium/high/critical)
v
ML Service returns:
| { theft_probability, anomaly_flag, anomaly_breakdown{8 scores},
|   source, confidence, risk_level, model_scores[], shap[] }
v
Backend: If anomaly_flag == true
| -> Create Alert in MongoDB with full breakdown
v
Frontend: Display result
| -> Anomaly: Red banner + 8 animated progress bars
| -> Clean: Green "All Clear" banner + toast notification

```

17. Risk Classification -- 4 Tiers

Risk Level	Probability Range	Color	Dashboard Action
Low	0% - 29%	Green	No action needed
Medium	30% - 49%	Amber	Schedule inspection within 30 days
High	50% - 74%	Orange	Schedule inspection within 7 days
Critical	75% - 100%	Red	Immediate field investigation

Why 30% threshold (not 50%)?

- * Cost-benefit analysis: Missing a thief costs Rs.50,000+/year. A false alarm costs Rs.500 (one inspection).
- * Ratio: 100:1 -- it's 100x more expensive to miss a thief than to investigate a false alarm

* So we set the bar LOW to catch as many thieves as possible

18. Summary to Memorize

"We built PowerGuard, a full-stack ML-powered electricity theft detection system. It analyzes 6-dimensional smart meter telemetry (consumption, voltage, current, power factor, frequency, tamper flags) using two parallel detection approaches: 1) An 8-measure weighted heuristic engine that produces explainable per-factor scores (consumption drop 25%, tamper detection 20%, current anomaly 15%, etc.) with multi-measure boosting logic. 2) A 5-model ML ensemble (Random Forest, XGBoost, DNN, LSTM, Isolation Forest) combined via soft voting and stacking, trained on the SGCC benchmark dataset (42,372 consumers) with SMOTE for class imbalance handling and SHAP for explainability. We engineered 47+ features across 6 categories (statistical, temporal, anomaly, comparative, metadata, advanced) and achieved strong detection performance with 5-fold cross-validation. The system runs as a 4-service microservice architecture (React + Node.js + FastAPI + MongoDB) with a triple-layer fallback (ML ensemble -> Python heuristic -> JavaScript heuristic) ensuring zero downtime. The dashboard provides operators with real-time risk classification across 4 tiers and one-click bulk analysis of entire meter networks."

Interview tip: Don't memorize this word-for-word. Understand the WHY behind every decision. For any component, you should naturally explain: What? -> Why? -> How does it work technically? -> What alternatives did you consider? -> What's the trade-off?